

Deep Learning Winter term 25/26 – Exercise Sheet 9

Solutions (Tasks 1–3)

1) Filters and weight matrices

Notation

Input (3×3):

$$I = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

Filter (2×2):

$$W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix}$$

Flatten the input in row-major order:

$$\mathbf{x} = [a \ b \ c \ d \ e \ f \ g \ h \ i]^\top.$$

Convolution outputs (valid, stride = 1)

Applying the 2×2 filter over the 3×3 image (valid, stride = 1) yields a **2×2 feature map**—four hidden units:

$$\textbf{Top-left (patch } I_{1:2,1:2}\text{): } y_1 = w_{11} a + w_{12} b + w_{21} d + w_{22} e.$$

$$\textbf{Top-right (patch } I_{1:2,2:3}\text{): } y_2 = w_{11} b + w_{12} c + w_{21} e + w_{22} f.$$

$$\textbf{Bottom-left (patch } I_{2:3,1:2}\text{): } y_3 = w_{11} d + w_{12} e + w_{21} g + w_{22} h.$$

$$\textbf{Bottom-right (patch } I_{2:3,2:3}\text{): } y_4 = w_{11} e + w_{12} f + w_{21} h + w_{22} i.$$

Collect $\mathbf{y} = [y_1, y_2, y_3, y_4]^\top$.

Equivalent weight matrix (fully-connected view)

There exists a 4×9 matrix W_{fc} such that $\mathbf{y} = W_{\text{fc}}\mathbf{x}$. Placing the same 2×2 filter weights at the indices of each 2×2 patch, we get:

$$W_{\text{fc}} = \begin{bmatrix} w_{11} & w_{12} & 0 & w_{21} & w_{22} & 0 & 0 & 0 & 0 \\ 0 & w_{11} & w_{12} & 0 & w_{21} & w_{22} & 0 & 0 & 0 \\ 0 & 0 & 0 & w_{11} & w_{12} & 0 & w_{21} & w_{22} & 0 \\ 0 & 0 & 0 & 0 & w_{11} & w_{12} & 0 & w_{21} & w_{22} \end{bmatrix}.$$

Multiplying W_{fc} by $\mathbf{x}$ reproduces (y_1, y_2, y_3, y_4) above. This shows how a **single linear layer** with structured sparsity implements the same computation as a valid 2×2 convolution over a 3×3 input—i.e., **weight sharing** across locations via the repeated pattern of $\{w_{11}, w_{12}, w_{21}, w_{22}\}$.

2) Convolution

Given in sheet:

- Discrete 2D convolution:

$$h(i, j) = (f * g)(i, j) = \sum_x \sum_y f(x, y) g(i - x, j - y).$$

- Convolutional-layer input (no bias) for filter $W \in \mathbb{R}^{M \times M}$ and image I :

$$z_{in,i,j} = \sum_{m=1}^M \sum_{n=1}^M I_{i+m-1, j+n-1} w_{m,n}.$$

(As written, this is the **cross-correlation** form commonly used in CNN libraries—note there is no kernel flip.)

Task

Define f and g so that $z_{in,i,j} = h(i, j)$.

Solution

To express the cross-correlation-like expression as a true convolution (which flips the kernel), define:

- Kernel function f as the **flipped** filter:

$$f(x, y) = w_{M-x+1, M-y+1} \quad \text{for } x, y \in \{1, \dots, M\}.$$

- Signal g as a **shifted** version of the image:

$$g(u, v) = I_{u+M, v+M}.$$

Then

$$\begin{aligned} h(i, j) &= \sum_{x=1}^M \sum_{y=1}^M f(x, y) g(i - x, j - y) \\ &= \sum_{x=1}^M \sum_{y=1}^M w_{M-x+1, M-y+1} I_{i-x+M, j-y+M}. \end{aligned}$$

With the change of variables $m = M - x + 1$, $n = M - y + 1$ (so $x = M - m + 1$, $y = M - n + 1$), this becomes

$$\sum_{m=1}^M \sum_{n=1}^M w_{m,n} I_{i+m-1, j+n-1} = z_{in,i,j}.$$

Therefore, with the above f and g , we obtain $h(i, j) = z_{in,i,j}$, i.e., the CNN pre-activation equals the mathematical convolution at the same indices. (Equivalently, you can keep $g = I$ and write $z_{in,i,j} = (\tilde{W} * I)(i + M - 1, j + M - 1)$ where $\tilde{W}$ is the 180° -rotated kernel.)

3) Feature Matching

Setup from sheet:

- A CNN for 5×5 images, no biases, hidden units are linear.
- A specific training case shows a “3” (black pixels = 1, white = 0).
- A 4×4 filter with weights

$$\begin{aligned} w_{11} &= 1, w_{12} = 1, w_{13} = 1, w_{14} = 0, \\ w_{21} &= 0, w_{22} = 0, w_{23} = 1, w_{24} = 0, \\ w_{31} &= 1, w_{32} = 1, w_{33} = 1, w_{34} = 0, \\ w_{41} &= 0, w_{42} = 0, w_{43} = 1, w_{44} = 0, \end{aligned}$$

and **valid, stride = 1** mapping from 5×5 to a 2×2 feature map (four hidden units).

What to compute

For the given “3” input image, compute the output of **each** of the four hidden units:

- z_{11} for the **top-left** 4×4 patch $I_{1:4, 1:4}$.
- z_{12} for the **top-right** 4×4 patch $I_{1:4, 2:5}$.
- z_{21} for the **bottom-left** 4×4 patch $I_{2:5, 1:4}$.
- z_{22} for the **bottom-right** 4×4 patch $I_{2:5, 2:5}$.

General formulas (binary image: 0/1)

Let $I_{ij} \in \{0, 1\}$. Each output is the sum of the 4×4 patch values where the filter has 1s:

$$\text{Top-left : } z_{11} = \underbrace{(I_{11} + I_{12} + I_{13})}_{\text{row 1, cols 1-3}} + \underbrace{(I_{23})}_{\text{row 2, col 3}} + \underbrace{(I_{31} + I_{32} + I_{33})}_{\text{row 3, cols 1-3}} + \underbrace{(I_{43})}_{\text{row 4, col 3}} .$$

$$\text{Top-right : } z_{12} = (I_{12} + I_{13} + I_{14}) + (I_{24}) + (I_{32} + I_{33} + I_{34}) + (I_{44}).$$

$$\text{Bottom-left : } z_{21} = (I_{21} + I_{22} + I_{23}) + (I_{33}) + (I_{41} + I_{42} + I_{43}) + (I_{53}).$$

$$\text{Bottom-right : } z_{22} = (I_{22} + I_{23} + I_{24}) + (I_{34}) + (I_{42} + I_{43} + I_{44}) + (I_{54}).$$

Since $w_{14}, w_{24}, w_{34}, w_{44} = 0$ (last column zero) and some interior zeros exist, only the entries shown above contribute. Numerical values depend on **which pixels are black (1) in the provided “3” image**.

Why this works: With a linear, bias-free unit, the pre-activation equals the sum of the patch pixels weighted by the filter. For a binary image, this is simply the **count** of black pixels at filter-1 positions within that patch. The four hidden units correspond to the four valid placements of the 4×4 filter over the 5×5 input.